



Data Structures And Algorithms

Name : Samreen Zia
Submitted to : Mr. Rehan Ahmed
Semester : BSE (3A)
Date : 27- Oct- 2025

LAB NO 5

Question no 1

Give answers to the following:

1. Show the contents of a (linear) queue and position of front and rear markers (at each step) once the following sequence of statements is executed. Queue Q;

Step	Operation	Queue Contents	Front	Rear
1.	Q.enqueue (10)	[10]	0	0
2.	Q.enqueue (20)	[10, 20]	0	1
3.	Q.enqueue (30)	[10, 20, 30]	0	2
4.	Q.enqueue ()	[20, 30]	1	2
5.	Q.enqueue ()	[30]	2	2
6.	Q.enqueue (40)	[30, 40]	2	3
7.	Q.enqueue ()	[40]	3	3
8.	Q.enqueue ()	[]	-1	

2. Consider a circular QUEUE with N=8 memory cells. Find the number of elements in QUEUE for the following positions of front and rear.

- $\text{front} = 0; \text{rear} = 4;$

$$\text{Number of elements} = (\text{rear} - \text{front} + N) \% N$$

$$= (4 - 0 + 8) \% 8$$

$$= 4$$
- $\text{front} = 2; \text{rear} = 0;$

$$\text{Number of elements} = (\text{rear} - \text{front} + N) \% N$$

$$= (0 - 2 + 8) \% 8$$

$$= 6$$
- $\text{front} = 4; \text{rear} = 6,$ and two elements are dequeued.

$$\text{Initially} : (6 - 4 + 8) \% 8$$

$$= 2 \text{ elements}$$

After dequeuing 2 elements $\rightarrow$

$$2 - 2 = 0$$

 (empty Queue)

3. Suppose q is an instance of a circular queue and the queue size is 4. Show the contents of the queue and positions of the front and rear markers once the following sequence of statements is executed. The initial contents of the queue are listed in the following.

Step	Operation	Queue	front	Rear
Initial	[40, 60, 180]	0	3	
1	dequeue() $\rightarrow$ remove 40	[, 60, 80]	1	3
2	dequeue() $\rightarrow$ remove 60	[, , 80]	2	3
3	enqueue(15)	[, , 15, 80]	2	0
4	enqueue(25)	[25, , 15, 80]	2	1
5	enqueue(105)	[25, 105, 15, 180]	2	

Task # 01

Create a class Queue that implements the functionality of a queue providing all the required operations (Addqueue(), Removequeue(), is_Empty(), is_Full(), display() , getFront(), getRare()).

Program:

```
#include <iostream>
using namespace std;
```

```

class Queue {
private:
    int *arr;
    int front, rear;

public:
    Queue(int n) {
        arr = new int[n];
        front = rear = -1;
    }

    ~Queue() {
        delete[] arr;
    }

    void Addqueue(int val) {
        if (is_Empty()) {
            front = rear = 0;
            arr[rear] = val;
        } else if (is_Full()) {
            cout << "Queue is full." << endl;
            return;
        } else {
            rear = (rear + 1) % 5; // assuming queue size is 5 for simplicity
            arr[rear] = val;
        }
        cout << "Added " << val << " to queue." << endl;
    }

    void Removequeue() {
        if (is_Empty()) {
            cout << "Queue is empty." << endl;
            return;
        }
        cout << "Removed " << arr[front] << " from queue." << endl;
        if (front == rear) {
            front = rear = -1;
        } else {
            front = (front + 1) % 5; // assuming queue size is 5 for simplicity
        }
    }

    bool is_Empty() {
        return front == -1;
    }
}

```

```

bool is_Full() {
    return (rear + 1) % 5 == front; // assuming queue size is 5 for simplicity
}

void display() {
    if (is_Empty()) {
        cout << "Queue is empty." << endl;
        return;
    }
    cout << "Queue elements: ";
    int i = front;
    do {
        cout << arr[i] << " ";
        if (i == rear) break;
        i = (i + 1) % 5; // assuming queue size is 5 for simplicity
    } while (i != front); // adjusted loop condition
    cout << endl;
}

int getFront() {
    if (is_Empty()) {
        cout << "Queue is empty." << endl;
        return -1;
    }
    return arr[front];
}

int getRare() {
    if (is_Empty()) {
        cout << "Queue is empty." << endl;
        return -1;
    }
    return arr[rear];
}

};

int main() {
    Queue q(5);
    q.Addqueue(1);
    q.Addqueue(2);
    q.display();
    cout << "Front element: " << q.getFront() << endl;
    cout << "Rear element: " << q.getRare() << endl;
}

```

```
q.Removequeue();
q.display();
return 0;
}
```

Output:

```
Added 1 to queue.
Added 2 to queue.
Queue elements: 1 2
Front element: 1
Rear element: 2
Removed 1 from queue.
Queue elements: 2
```

Task # 2:

Create a class Circular Queue that implements the functionality of a queue providing all the required operations (Addqueue(), Removequeue(), Empty(), Full() and getFront()).

Program:

```
#include <iostream>
using namespace std;

class CircularQueue {
private:
    int *arr;
    int front, rear;

public:
    CircularQueue(int n) {
        arr = new int[n];
        front = rear = -1;
    }

    ~CircularQueue() {
        delete[] arr;
    }

    void Addqueue(int val) {
        if (is_Full()) {
            cout << "Queue is full." << endl;
            return;
        }
    }
}
```

```

    }
    if (is_Empty()) {
        front = rear = 0;
    } else {
        rear = (rear + 1) % 5; // assuming queue size is 5 for simplicity
    }
    arr[rear] = val;
    cout << "Added " << val << " to queue." << endl;
}

void Removequeue() {
    if (is_Empty()) {
        cout << "Queue is empty." << endl;
        return;
    }
    cout << "Removed " << arr[front] << " from queue." << endl;
    if (front == rear) {
        front = rear = -1;
    } else {
        front = (front + 1) % 5; // assuming queue size is 5 for simplicity
    }
}

bool is_Empty() {
    return front == -1;
}

bool is_Full() {
    return (rear + 1) % 5 == front; // assuming queue size is 5 for simplicity
}

int getFront() {
    if (is_Empty()) {
        cout << "Queue is empty." << endl;
        return -1;
    }
    return arr[front];
}
};

int main() {
    CircularQueue q(5);
    q.Addqueue(1);
    q.Addqueue(2);

```

```

    cout << "Front element: " << q.getFront() << endl;
    q.Removequeue();
    cout << "Front element: " << q.getFront() << endl;
    return 0;
}

```

Output:

```

Added 1 to queue.
Added 2 to queue.
Front element: 1
Removed 1 from queue.
Front element: 2

```

Task # 3:

Use Stack AND Queue combination to find whether a word is a PALINDROME or not. Use built in stack and queue.

Program:

```

#include <iostream>
#include <string>

```

```

class Stack {
public:
    char data[100];
    int top;

    Stack() { top = -1; }

    void push(char c) { data[++top] = c; }
    char pop() { return data[top--]; }
    bool isEmpty() { return top == -1; }
};

```

```

class Queue {
public:
    char data[100];
    int front, rear;

    Queue() { front = rear = -1; }

```

```

void enqueue(char c) {
    if (rear == -1) front = rear = 0;
    else rear++;
    data[rear] = c;
}
char dequeue() { return data[front++]; }
bool isEmpty() { return front == -1 || front > rear; }
};

bool isPalindrome(const std::string& word) {
    Stack s;
    Queue q;
    for (char c : word) {
        s.push(c);
        q.enqueue(c);
    }
    while (!s.isEmpty()) {
        if (s.pop() != q.dequeue()) return false;
    }
    return true;
}

int main() {
    std::string word;
    std::cout << "Enter a word: ";
    std::cin >> word;

    std::cout << word << (isPalindrome(word) ? " is" : " is not") << " a palindrome." << std::endl;

    return 0;
}

```

Output:

```

Enter a word: madam
madam is a palindrome.

Enter a word: hello
hello is not a palindrome.

```